

# Weight Initialization

## A Beginner's Companion Guide

---

*How the very first numbers in a network decide whether it learns at all*

---

# About This Guide

## Why the Starting Point Matters Before Any Learning Happens

*A plain-language walkthrough of zeros, large-random, and He initialization — and the questions they answer*

Before a neural network can learn anything, it has to start somewhere. Every weight in the network needs an initial value, set the moment the network is built and before it has seen a single example. It is tempting to think this starting point hardly matters — surely training will sort it out? This guide shows, with three short functions you wrote yourself, that the opposite is true: the same network, trained for the same number of steps with the same settings, can fail completely or succeed brilliantly depending on nothing more than how its weights were first filled in.

You experimented with three initialization methods on a small task: separating blue dots from red dots arranged in two interlocking circles, using a three-layer network. The three methods — **zeros**, **large random values**, and **He initialization** — lead to dramatically different outcomes. This guide explains each one in plain terms, defines the key concepts behind it, and answers the practical questions the assignment raises: how to choose an initialization, how a good one speeds learning and reaches lower error, and the central puzzle — if large random weights are bad and zeros are bad, exactly how small should the random values be?

### HOW TO READ THIS GUIDE

Each section follows the same rhythm: the code you wrote, what it does in everyday language, the key concept defined, and a short note on where the idea lives in the wider world of deep learning. You need no mathematics beyond multiplication, division, and the idea of a square root. The three methods build on one another, so it is worth reading them in order.

## The Question Behind It All

If you have trained networks before, you may have used a weight-initialization recipe without thinking about it, and it simply worked. So the natural question is the one the assignment opens with: **but how do you choose the initialization for a new neural network?** The honest answer is that you choose it by understanding what can go wrong, and a well-chosen initialization does two concrete things for you. It can **speed up the convergence of gradient descent** — the network reaches a good solution in far fewer steps — and it can **increase the odds of gradient descent converging to a lower training and generalization error**, meaning the network ends up more accurate, both on the data it trained on and on new data it has never seen. The three experiments below show exactly how, and why.

### KEY CONCEPT: INITIALIZATION

Initialization is the act of assigning starting values to a network's weights and biases before training begins. Training adjusts these values step by step using gradient descent, but it can only adjust what it starts with. A poor starting point can leave the network unable to improve at all, or force it to waste thousands of steps

digging out of a bad spot. The initialization is not part of the learning — it is the launchpad the learning fires from.

## 1. Zero Initialization — The Tempting Mistake

The simplest possible idea is to set every weight and every bias to zero. The function fills each weight matrix and bias vector with zeros of the correct shape:

```
# GRADED FUNCTION: initialize_parameters_zeros

def initialize_parameters_zeros(layers_dims):
    parameters = {}
    L = len(layers_dims)          # number of layers in the network

    for l in range(1, L):
        parameters['W' + str(l)] = np.zeros((layers_dims[l], layers_dims[l-1]))
        parameters['b' + str(l)] = np.zeros((layers_dims[l], 1))
    return parameters
```

The shapes are worth pausing on, because they are the same in all three methods. A weight matrix **W** for layer **l** has shape `(layers_dims[l], layers_dims[l-1])` — one row per neuron in the current layer, one column per input coming from the previous layer. A bias vector **b** has shape `(layers_dims[l], 1)` — one bias per neuron. Only the *values* change between methods; the shapes never do.

Trained on the dots, this network reaches just **50% accuracy** — no better than flipping a coin. The cost barely moves across all 15,000 iterations, and the network ends up predicting the same answer for every single input. To see why, follow what happens to one example. With all weights zero, every neuron computes the same thing: zero times anything is zero, so each neuron's input **z** is zero. The ReLU of zero is zero, so every hidden activation is zero too. At the final layer the sigmoid of zero is exactly 0.5, so the network predicts 0.5 for every example, regardless of whether the true answer is red or blue.

### KEY CONCEPT: BREAKING SYMMETRY

When every neuron in a layer starts with identical values, they all compute the same output and therefore all receive the *same* correction signal (gradient) during training. They update in lockstep and stay identical forever — perfect clones. A layer of 10 identical neurons can only ever learn what a single neuron could. **Breaking symmetry** means giving neurons different starting values so they can specialize and learn different features. Zero initialization fails precisely because it never breaks symmetry.

There is a subtle but important detail here. The problem is not the number zero itself — it is *sameness*. Initializing every neuron to the same non-zero value, say 0.01, fails in exactly the same way: the neurons are still identical, so they still learn identically. The real enemy is identical initialization across neurons. This is also why biases are allowed to start at zero: as long as the *weights* differ from neuron to neuron, symmetry is already broken, and the biases can safely begin at zero without causing any harm.

### WHERE THIS MAPS IN DEEP LEARNING

Foundation: every modern framework initializes weights randomly by default precisely to avoid this trap — you simply inherit the lesson of this exercise. Advanced: the symmetry insight explains why a network's *width* (its number of neurons per layer) only buys you power if those neurons can diverge. It is also why logistic regression, which has a single output unit and no hidden layer, can train fine from zero — there is no symmetry to break in the first place.

## 2. Large Random Initialization — Breaking Symmetry, Badly

The fix for symmetry is randomness. If each weight starts at a different random value, each neuron begins life differently and can learn its own distinct feature. This function draws random numbers from a normal distribution and — deliberately, to make a point — scales them up by a large factor of 10:

```
# GRADED FUNCTION: initialize_parameters_random

def initialize_parameters_random(layers_dims):
    np.random.seed(3)                # fix the "random" numbers
    parameters = {}
    L = len(layers_dims)              # number of layers

    for l in range(1, L):
        parameters['W' + str(l)] = np.random.randn(layers_dims[l], layers_dims[l-1]) * 10
        parameters['b' + str(l)] = np.zeros((layers_dims[l], 1))
    return parameters
```

The only change from the zeros version is the weight line: `np.random.randn(...) * 10` draws a grid of random values and multiplies each by 10. The biases still start at zero, which is fine now that the weights are random. The `np.random.seed(3)` line simply fixes the sequence of "random" numbers so your results are reproducible — run it twice and you get the same starting weights.

This network reaches **83% accuracy** — far better than the coin-flip of zeros, because symmetry is now broken and the neurons genuinely differ. Progress! But the cost starts *enormously* high (you may even see `inf` on the first iteration), and learning is sluggish. The culprit is the factor of 10, which makes the weights far too large.

### WHY THIS MATTERS: SATURATION AND EXPLODING LOSS

When weights are large, the value fed into the final sigmoid is large too, so the sigmoid output is pushed hard against 0 or 1 — it becomes extremely confident. When such a confident prediction is *wrong*, the loss function, which contains a logarithm, blows up: the log of a number near zero races toward negative infinity, producing that `inf` cost. Worse, in the flat regions near 0 and 1 the sigmoid's slope is almost zero, so the gradient nearly vanishes and the weights barely budge. The network has to crawl out of this hole before real learning can begin.

### KEY CONCEPT: RANDOM DISTRIBUTIONS

The code uses `np.random.randn`, which draws from a **normal** (Gaussian) distribution — most values cluster near zero, with fewer far out. This differs from `np.random.rand`, which spreads values evenly across a fixed range. The normal distribution is preferred for weights because it keeps most of them modest and near the center, which — with the right scale — helps activations avoid the flat, slow-learning extremes of the sigmoid.

This experiment answers the first half of the assignment's central question. We have learned, concretely, that **initializing weights to very large random values does not work well**. Randomness was necessary to break symmetry, but the *scale* of that randomness turned out to matter just as much. Small random values should do better — which raises the question the next method finally answers: how small?

### 3. He Initialization — Randomness at the Right Scale

He initialization keeps the randomness that breaks symmetry but replaces the arbitrary factor of 10 with a scale carefully calculated from the size of each layer. The only change from the previous function is the multiplier:

```
# GRADED FUNCTION: initialize_parameters_he

def initialize_parameters_he(layers_dims):
    np.random.seed(3)
    parameters = {}
    L = len(layers_dims) - 1      # integer representing the number of layers

    for l in range(1, L + 1):
        parameters['W' + str(l)] = (np.random.randn(layers_dims[l], layers_dims[l-1])
                                     * np.sqrt(2. / layers_dims[l-1]))
        parameters['b' + str(l)] = np.zeros((layers_dims[l], 1))
    return parameters
```

Instead of `* 10`, the weights are multiplied by `np.sqrt(2. / layers_dims[l-1])` — the square root of two divided by the number of inputs feeding into the layer. (The loop is written slightly differently, with `L = len(layers_dims) - 1` and `range(1, L + 1)`, but it covers exactly the same layers; use `2.` with the decimal point to keep the division in floating point.) Trained on the dots, this network reaches **99% accuracy** and separates the colors cleanly in a small number of iterations. Same network, same data, same number of steps — only the starting scale changed.

#### THE ANSWER: HOW SMALL SHOULD THE RANDOM VALUES BE?

This is the question the whole assignment builds toward. The answer is not a fixed number but a *rule tied to the size of each layer*: the more inputs a neuron receives, the smaller each weight should be. The reasoning is simple. A neuron adds up many `weight × input` terms; the more terms in that sum, the larger it tends to grow. To keep the sum in a healthy mid-range — not so large it saturates, not so small it fades — you shrink each weight as the input count rises. He initialization encodes exactly this by setting the weight scale to the square root of `2 / (number of inputs to the layer)`.

### KEY CONCEPT: VANISHING AND EXPLODING GRADIENTS

In a deep network, the correction signal is multiplied layer by layer as it travels backward during training. If the weights are consistently a little too large, these multiplications compound and the signal **explodes** toward huge values; if a little too small, it **vanishes** toward zero, and the early layers stop learning. Both wreck training. Keeping the weight scale calibrated to each layer's size — what He initialization does — keeps the signal at a stable, useful magnitude as it passes through, which is why it converges so much faster.

The "2" in the formula is specific to the **ReLU** activation used in the hidden layers here. A closely related recipe, Xavier initialization, uses a "1" instead of a "2" and suits activations like tanh; He et al. showed in 2015 that the factor of 2 works better for ReLU networks, because ReLU zeroes out roughly half its inputs and the extra factor compensates for that lost signal. The practical takeaway is simple: match the initialization to the activation, and for the ReLU networks you will build most often, He initialization is the dependable default.

### WHERE THIS MAPS IN DEEP LEARNING

Foundation: He initialization (and its cousin Xavier) is the standard starting point baked into every major framework's ReLU layers — the recipe you just wrote by hand is the one running underneath. Advanced: the same goal of keeping signal magnitudes stable across depth motivated later inventions like batch normalization and the skip connections in residual networks (ResNets), which let networks grow to hundreds of layers without the signal vanishing or exploding. Good initialization was the first step on that road.

## 4. The Three Methods Side by Side

For the same network, the same data, the same number of iterations, and the same learning rate, the only difference between these three runs was the starting values of the weights — yet the results could hardly be more different:

| Model                                   | Train Accuracy | Problem / Comment       |
|-----------------------------------------|----------------|-------------------------|
| 3-layer NN, zeros initialization        | 50%            | Fails to break symmetry |
| 3-layer NN, large random initialization | 83%            | Weights too large       |
| 3-layer NN, He initialization           | 99%            | Recommended method      |

Read top to bottom, the table tells the whole story. Zeros never break symmetry, so the network is stuck at a coin flip. Large random values break symmetry but start the network in a saturated, slow region, leaving it at a mediocre 83% after the same training budget. He initialization breaks symmetry *and* starts at the right scale, reaching 99% in the same number of steps. This is the assignment's two promises made visible: a well-chosen initialization **speeds convergence** (He reaches a great solution quickly) and **reaches a lower error** (99% versus 83% versus 50%), on both the training data and new data.

## 5. What to Remember

So, how do you choose the initialization for a new neural network? The four lessons from these experiments give a complete, practical answer:

#### THE MAIN TAKEAWAYS

**Different initializations lead to very different results.** The starting point is not a detail training will fix; it can decide whether the network learns at all.

**Random initialization breaks symmetry**, letting different neurons learn different things. Without it, a wide layer is no better than a single neuron. Biases, however, may safely start at zero.

**Resist initializing to values that are too large.** Oversized weights saturate the activations, spike the loss, and slow learning to a crawl.

**He initialization works well for networks with ReLU activations.** By scaling weights to the square root of  $2 / (\text{inputs to the layer})$ , it breaks symmetry at just the right magnitude — the dependable default for the networks you will build most.

The deepest lesson hides in plain sight. Three functions that differ by a single multiplier — nothing, ten, or a square root — produced a network that could not learn, one that learned poorly, and one that learned almost perfectly. Long before backpropagation takes its first step, the numbers you start with have already shaped what the network can become. That is why initialization, modest as it looks, is one of the quiet foundations everything else in deep learning is built upon.